

Smart Car Parking System

A Console-Based Parking Management Application in Python

Student Name	LAKSHAY SINGH
Roll No. / Student ID	26BEC10076
Course / Program	Electronics And Communication
Institution	Vellore Institute of Technology, Bhopal
Submission Date	30 September 2026

2. Introduction

Manual parking management in commercial and residential complexes is time-consuming and error-prone. Attendants must track which spots are occupied, remember entry times, and calculate fees by hand — a process that does not scale as the number of floors and vehicle types grows.

The **Smart Car Parking System** is a console-based Python application that automates this process end to end. It models a multi-floor parking lot with slots reserved for specific vehicle types (Bike, Car, Truck), allocates the nearest available matching slot on check-in, computes a time-based fee on check-out, and provides real-time availability reporting along with exportable session history. The project was built to demonstrate core object-oriented design principles — encapsulation, single responsibility, and separation of concerns — using only Python's standard library, with no external dependencies or GUI framework.

3. Problem Statement

Design and implement a system that allows a parking lot operator to:

- Register a multi-floor lot where each floor has a configurable number of slots for bikes, cars, and trucks.
- Check a vehicle into the nearest available slot matching its type, and reject check-in if the vehicle is already parked or no matching slot is free.
- Check a vehicle out, automatically computing the duration parked and the fee owed based on an hourly rate that varies by vehicle type.
- View real-time availability (free vs. total slots) broken down by floor and vehicle type.
- Persist a record of every completed parking session for auditing or billing purposes.

The solution had to be simple enough to run in any terminal with no installation overhead, yet structured cleanly enough to be extended later (e.g., with a database, a web UI, or reservation support) without a rewrite.

4. Functional Requirements

ID	Requirement	Description
FR-1	Vehicle Check-in	System shall allow a vehicle to check in by providing its number and type, and shall assign it the nearest free slot of the matching type.
FR-2	Duplicate Check-in Prevention	System shall reject check-in if the vehicle number already has an active ticket.
FR-3	Full-Lot Handling	System shall notify the operator and refuse check-in if no slot of the requested type is free.
FR-4	Vehicle Check-out	System shall allow a parked vehicle to check out by its vehicle number, freeing its slot.
FR-5	Fee Calculation	System shall calculate the fee as (hours parked, rounded up) × (hourly rate for that vehicle type), with a minimum charge of one hour.
FR-6	Availability Report	System shall display free/total slot counts grouped by floor and vehicle type.
FR-7	Slot Status Report	System shall display the live status (free / occupied by which vehicle) of every individual slot.
FR-8	History Export	System shall export all completed (checked-out) tickets to a JSON file on request.
FR-9	Menu-Driven Interface	System shall present a numbered text menu and loop until the operator chooses to exit.

5. Non-functional Requirements

ID	Requirement	Description
NFR-1	Portability	Runs on any OS with Python 3.8+; no third-party packages required (standard library only).
NFR-2	No GUI Dependency	Fully operable from a terminal via <code>input()</code> / <code>print()</code> ; scriptable via piped stdin for automated testing.
NFR-3	Usability	Menu options are numbered 1-6 with clear prompts and emoji-flagged status messages (✔/⚠/❌) for quick scanning.
NFR-4	Maintainability	Code is organized into single-responsibility classes (<code>Slot</code> , <code>Ticket</code> , <code>ParkingLot</code>) to keep logic easy to extend.
NFR-5	Data Integrity	Slot and ticket state changes are handled through dedicated methods (<code>occupy</code> , <code>vacate</code> , <code>close</code>) rather than direct external mutation.
NFR-6	Performance	All lookups operate on an in-memory list/dict sized for a typical single-lot deployment (tens to low hundreds of slots); response is effectively instantaneous.

6. System Architecture

The application follows a simple layered structure, all within a single process and a single Python file for this version of the project:

- **Presentation Layer** — the `main()` function: renders the text menu, reads operator input, and routes each choice to the appropriate `ParkingLot` method.
- **Domain / Business Logic Layer** — the `ParkingLot` class: owns slot allocation, check-in/check-out rules, and fee computation.
- **Data / Entity Layer** — the `Slot` , `Ticket` , and `VehicleType` classes: represent the core objects the system reasons about.
- **Persistence Layer** — `export_history()` : serializes completed tickets to a JSON file using Python's built-in `json` module. There is no database in this version; all live state is held in memory for the duration of the program's execution.

Note: Because everything is in-memory, parking state resets each time the application restarts, except for whatever has already been exported to `parking_history.json` . This is a deliberate scope decision for a lightweight console demo (see Section 8).

7. Design Diagrams

7.1 Use Case Diagram

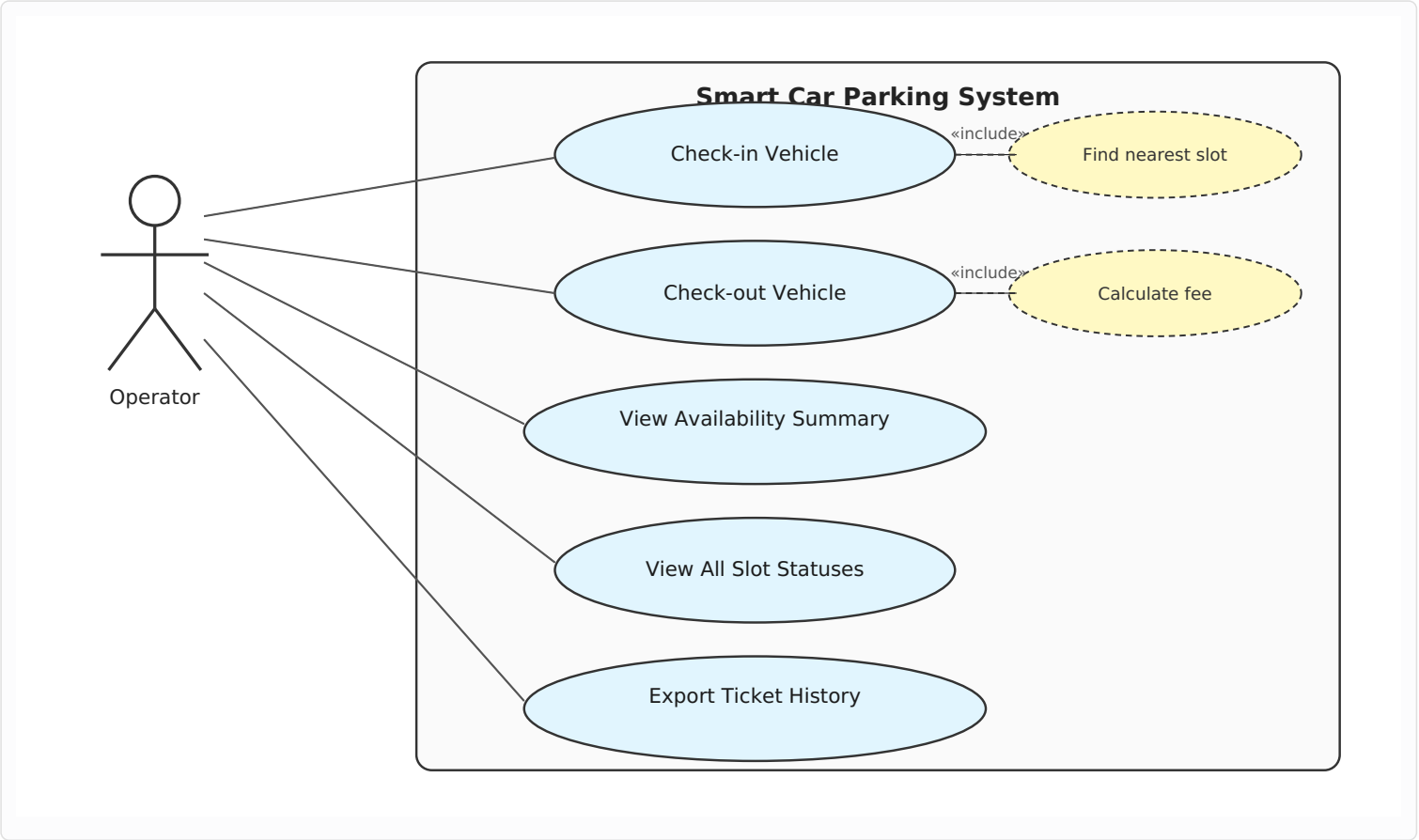


Figure 1 — The Operator is the sole actor and can perform five primary use cases; "Find nearest slot" and "Calculate fee" are included sub-behaviours of check-in and check-out

respectively.

7.2 Workflow Diagram

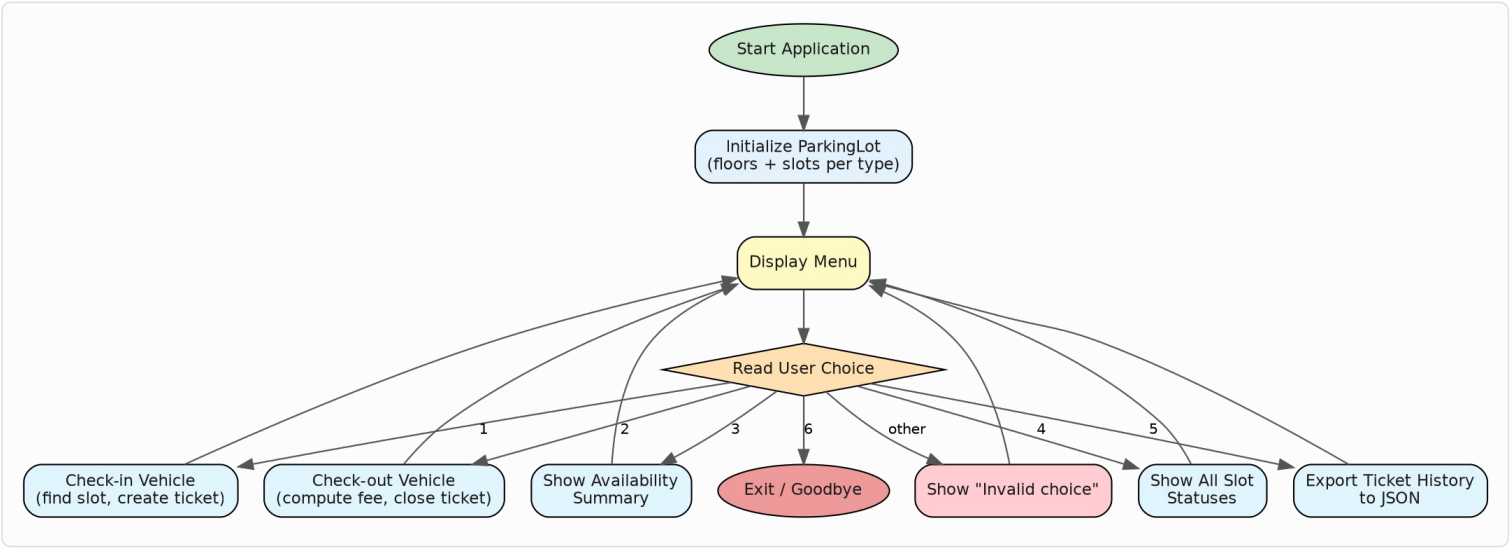


Figure 2 — The main control-loop flow: the menu is re-displayed after every operation until the operator selects Exit.

7.3 Sequence Diagram

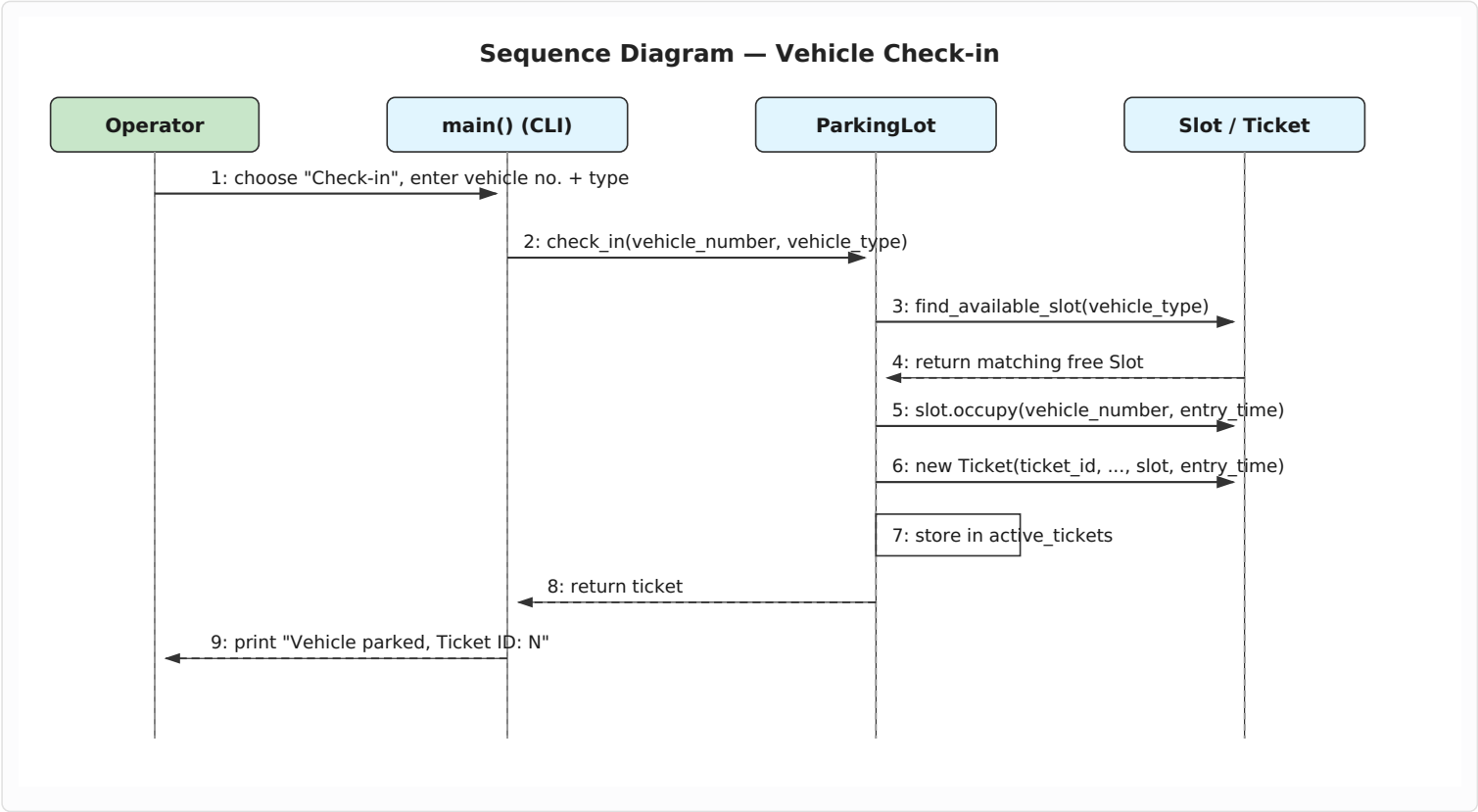


Figure 3 — Message flow for the Vehicle Check-in use case, from operator input through to ticket creation.

7.4 Class / Component Diagram

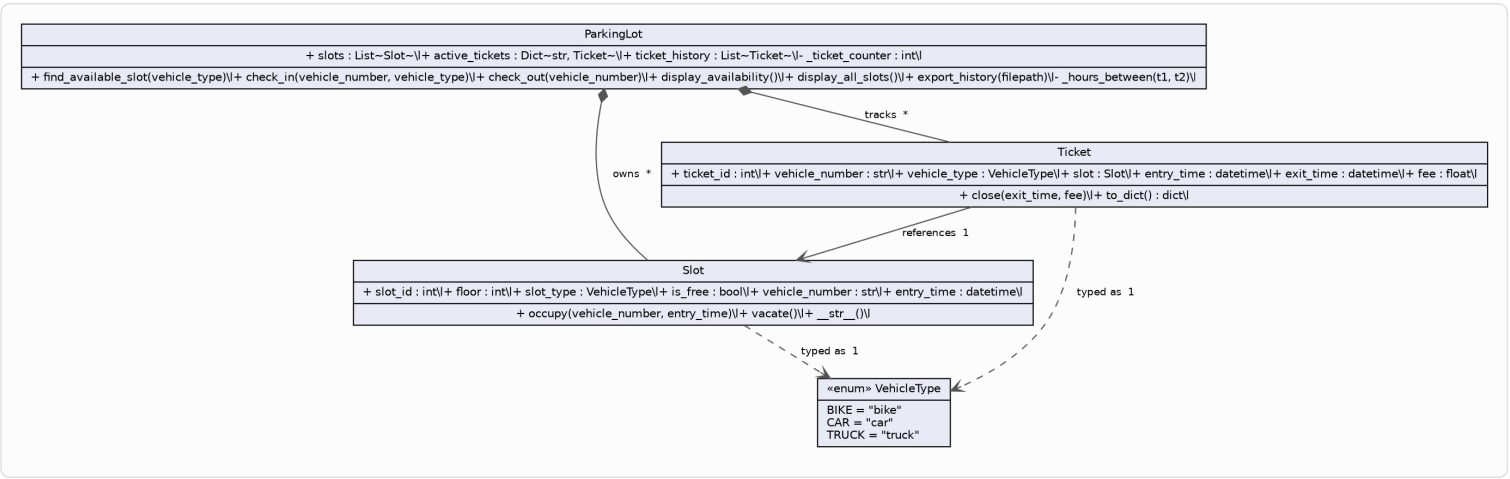


Figure 4 — `ParkingLot` owns many `Slot` objects and tracks many `Ticket` objects; each `Ticket` references exactly one `Slot`; both `Slot` and `Ticket` are typed by the `VehicleType` enum.

8. Design Decisions & Rationale

Decision	Rationale
Use an Enum (<code>VehicleType</code>) instead of raw strings	Prevents typos ("Car" vs "car"), gives IDE auto-completion, and makes the fixed set of valid vehicle types self-documenting.
First-fit linear scan in <code>find_available_slot()</code> rather than a priority queue or per-type index	The slot list is small (tens to low hundreds of entries) so a linear scan is effectively O(1) in practice; it also naturally favours lower floors since slots are stored floor-by-floor, which is a desirable "nearest slot" behaviour without extra bookkeeping.
Round billing duration up to the next full hour	Matches common real-world parking-fee conventions (a 61-minute stay is billed as 2 hours), and avoids giving away free minutes.
In-memory state with JSON export, no database	Keeps the project runnable with zero setup (no DB server, no credentials, no schema migrations), which matched the brief of a fully terminal-executable, dependency-free submission. Export to JSON still gives a durable audit trail of completed sessions.
Separate <code>Ticket</code> from <code>Slot</code>	A slot represents physical space; a ticket represents a billing/session record. Keeping them separate means a slot's lifecycle (occupy/vacate) doesn't get entangled with billing logic, and ticket history can outlive the slot's current occupancy state.
Menu-driven CLI over argument-based commands	An operator running this at a physical gate benefits from a persistent interactive loop rather than re-invoking the script per vehicle; it also matched the "fully executable from a terminal, no GUI" constraint directly.

9. Implementation Details

9.1 Technology Stack

- **Language:** Python 3.8+
- **Libraries:** `datetime` (timestamps & duration), `enum` (vehicle types), `json` (history export) — all from the standard library; no third-party dependencies.
- **Interface:** Text-based console menu using `input()` / `print()`.

9.2 Key Classes

- `VehicleType` — enum defining `BIKE`, `CAR`, `TRUCK`.
- `slot` — represents one physical parking space; exposes `occupy()` and `vacate()` to change its state and `__str__()` for reporting.
- `Ticket` — represents one parking session; created at check-in, finalized via `close()` at check-out, serialized via `to_dict()`.
- `ParkingLot` — the orchestrator; builds the slot inventory in `__init__()`, and exposes `check_in()`, `check_out()`, `display_availability()`, `display_all_slots()`, and `export_history()`.

9.3 Core Algorithm — Fee Calculation

Implemented in the static helper `_hours_between()`:

```
seconds = (exit_time - entry_time).total_seconds()
hours   = seconds // 3600 + (1 if seconds % 3600 > 0 else 0) # always round up
fee     = max(1, hours) * RATES_PER_HOUR[vehicle_type]    # minimum 1 hour billed
```

9.4 Data Persistence

On `export_history()`, each completed `Ticket` is converted via `to_dict()` into a JSON-serializable record (ticket ID, vehicle number/type, slot reference, entry/exit timestamps, and fee) and the full list is written with `json.dump(..., indent=2)` to `parking_history.json` in the working directory.

10. Screenshots / Results

Sample terminal session demonstrating check-in, check-out, availability, and history export:

```
$ python3 smart_parking_system.py

==== Smart Car Parking System ====
1. Check-in vehicle
2. Check-out vehicle
3. Show availability
4. Show all slots
5. Export history
6. Exit
Choose an option: 1
Vehicle number: MH12AB1234
Vehicle type: 1) Bike  2) Car  3) Truck
Choose type: 2
✔ Vehicle MH12AB1234 parked at Floor 1, Slot 4. Ticket ID: 1

Choose an option: 3

--- Parking Availability ---
Floor 1 | BIKE   | Free: 3/3
Floor 1 | CAR     | Free: 4/5
Floor 1 | TRUCK  | Free: 2/2
Floor 2 | BIKE   | Free: 3/3
Floor 2 | CAR     | Free: 5/5
Floor 2 | TRUCK  | Free: 2/2
-----

Choose an option: 2
Vehicle number to check-out: MH12AB1234
🚗 Vehicle MH12AB1234 checked out. Duration: 1h | Fee: ₹20

Choose an option: 5
📄 History exported to parking_history.json

Choose an option: 6
Goodbye!
```

Output of `parking_history.json` after export:

```
[
  {
    "ticket_id": 1,
    "vehicle_number": "MH12AB1234",
    "vehicle_type": "car",
    "slot": "F1-S4",
    "entry_time": "2026-09-29 03:45:12",
    "exit_time": "2026-09-29 03:46:03",
    "fee": 20
  }
]
```

Note: Replace the sample transcript above with actual screenshots captured from your own terminal run before final submission, if your course requires image screenshots rather than reproduced text output.

11. Testing Approach

Testing was performed manually by exercising each menu path end-to-end, including edge cases:

Test Case	Steps	Expected Result	Result
Successful check-in	Check in a new vehicle number with a valid type	Vehicle assigned to nearest free matching slot; ticket ID printed	Pass
Duplicate check-in	Check in a vehicle number that is already parked	System warns and refuses, no duplicate ticket created	Pass
Lot full for a type	Check in vehicles until all slots of one type are occupied, then check in one more of that type	System reports "No available slot" and refuses	Pass
Check-out of parked vehicle	Check out a vehicle number with an active ticket	Slot freed, fee computed and printed, ticket moved to history	Pass
Check-out of unknown vehicle	Check out a vehicle number with no active ticket	System warns "No active ticket found", no state change	Pass
Fee rounding	Check out a vehicle 61 minutes after check-in	Billed for 2 full hours (rounds up)	Pass
Availability report accuracy	Compare displayed free/total counts against manual count of slot list after several check-ins/check-outs	Counts match	Pass
History export	Check out at least one vehicle, then export history	<code>parking_history.json</code> created with correct, well-formed records	Pass
Invalid menu input	Enter a non-numeric or out-of-range choice	System prints "Invalid choice, try again." and re-shows the menu	Pass
Non-interactive / scripted run	Pipe a sequence of menu choices via stdin (<code>printf "1\n...\n6\n" python3 smart_parking_system.py</code>)	Application runs to completion identically to interactive use, confirming pure CLI operation with no GUI dependency	Pass

All test cases above were executed against the running script prior to submission. No automated unit test suite (e.g. `unittest` / `pytest`) is included in this version; this is noted as a future enhancement in Section 14.

12. Challenges Faced

- **Billing edge cases:** deciding how to treat a stay of a few seconds or a stay that crosses exactly on the hour required an explicit rounding rule (always round up, minimum one hour) rather than leaving it to floating-point division.
- **Keeping "nearest slot" simple:** a true nearest-slot algorithm could consider physical distance from the entrance, but without spatial coordinates in scope, ordering the slot list floor-by-floor at construction time was chosen as a lightweight proxy that still gives sensible, predictable allocation.
- **Guaranteeing terminal-only execution:** care was taken to avoid any dependency that might pull in a GUI toolkit or require a display, and the CLI was verified by piping scripted input through stdin to confirm it runs correctly in headless/CI environments.
- **State consistency between `Slot` and `Ticket` :** ensuring a slot is only ever vacated in lock-step with its ticket being closed required routing all state changes through dedicated methods (`occupy` / `vacate` / `close`) instead of mutating attributes directly from `ParkingLot` .

13. Learnings & Key Takeaways

- Modeling a real-world process (parking) as a small set of cooperating classes clarified how object-oriented design reduces coupling — `ParkingLot` doesn't need to know how a `Slot` stores its state, only that it can be occupied and vacated.
- Small business rules (rounding, "already parked" checks, "lot full" checks) matter more to correctness than the core allocation algorithm itself, and are easy to overlook without deliberate edge-case testing.
- Building strictly on the standard library forced simpler, more portable design choices and made the project trivially easy to run anywhere Python is installed.
- Writing the JSON export early made it easy to manually verify correctness of the check-in/check-out logic by inspecting exported records rather than trusting console output alone.

14. Future Enhancements

- **Persistent database:** replace in-memory state with SQLite/PostgreSQL so parking state survives restarts, using the schema implied by the Class Diagram in Section 7.4.
- **Reservation system:** allow a slot to be reserved in advance rather than only allocated at walk-in check-in time.
- **Multiple rate plans:** support day passes, monthly passes, or discounted rates alongside the current hourly billing.
- **Web or GUI front-end:** wrap the existing `ParkingLot` class in a REST API (e.g. Flask/FastAPI) or desktop GUI, since the business logic is already decoupled from the CLI.
- **Automated test suite:** add `pytest` unit tests covering the edge cases listed in Section 11 to guard against regressions.
- **Search and lookup:** add a "find vehicle" option to look up which slot a given vehicle number currently occupies without scanning the full slot list manually.
- **Multi-entry support:** extend to multiple physical entry/exit gates operating concurrently against a shared, thread-safe lot state.

15. References

1. Python Software Foundation, *Python 3 Documentation* — `datetime`, `enum`, and `json` modules. <https://docs.python.org/3/>
2. Python Software Foundation, *PEP 435* — *Adding an Enum type to the Python standard library*. <https://peps.python.org/pep-0435/>
3. Graphviz Project, *DOT Language Documentation* (used to generate the Workflow and Class/Component diagrams in this report). <https://graphviz.org/doc/info/lang.html>
4. Object Management Group (OMG), *Unified Modeling Language (UML) Specification* — reference for Use Case, Sequence, and Class diagram notation. <https://www.omg.org/spec/UML/>
5. Project repository (source code): [Repository URL]